

Workflow Runtime Semantics

The **workflow** defines the internal sequence of actions that a skill performs to transform its input into a result. In the RAGenius Execution Subsystem, workflows are declarative and interpreted by the workflow orchestrator at runtime. This document describes the semantics of workflow steps, execution flow, error handling, retries and compensation.

Step Types

Each workflow is composed of a list of steps. Each step has a unique `id` and a `type` that determines how it is executed.

Type	Description	Typical usage
<code>validation</code>	Performs a local check on the execution context or input. It does not call external services. On failure, the workflow stops with a validation error.	Check field length, verify preconditions
<code>tool_call</code>	Invokes a registered tool through the tool engine. The step must specify <code>tool_id</code> and may include input and output mappings.	Call <code>rag_retrieval_tool</code> , <code>mock_video_generation_tool</code>
<code>local_decision</code>	Evaluates a boolean condition to decide which branch to follow. The condition can reference context values.	Dry run check
<code>internal_workflow</code>	Executes a nested workflow, allowing reuse of sub-flows or composition of common sequences.	Subroutine for generating a report
<code>service_call</code>	Calls an internal service (e.g. skill registry, permission engine). Generally used for system-level operations.	Load skill metadata, check permissions
<code>saga</code>	Defines a compensation sequence that runs if a preceding step fails after side effects.	Roll back file creation, delete external documents

Type	Description	Typical usage
<code>end</code>	Terminates the workflow. It may specify a status (e.g. <code>success</code> , <code>fail_validation</code>).	End states for success or failure

Step Structure

```

steps:
  - id: <string>
    type: <string>
    action: <string or object>
    input_mapping: <mapping>          # optional
    output_mapping: <mapping>        # optional
    retry:                          # optional
      max: <integer>
      backoff_ms: <integer>
    on:                              # optional transitions
      success: <step_id>
      failure: <step_id>
      partial: <step_id>
      requires_confirmation: <step_id>

```

Fields

- `id`: Unique identifier within the workflow. Use descriptive names such as `validate_prompt` or `generate_video`.
- `type`: Step type as defined above.
- `action`: For `tool_call` and `service_call`, this references the tool ID or binding reference. For `validation`, it can be the name of a local validation function. For `internal_workflow`, it references a sub-workflow definition.
- `input_mapping`: Maps values from the execution context to the tool or service input. Use [JMESPath](#) or JSON-pointer syntax to reference fields (e.g. `$.input.prompt` or `$.steps.retrieve_context.output.context`).
- `output_mapping`: Maps values from the tool or service output back into the execution context. Keys become available under `$.steps.<step_id>.output`.
- `retry`: Optional retry policy. `max` is the number of retries and `backoff_ms` is the delay between attempts. Retries apply only to `tool_call` and `service_call` types.
- `on`: Defines transitions based on outcome. `success`, `failure`, `partial`, or `requires_confirmation` point to the next step ID. If omitted, the workflow ends.

Input and Output Mapping

Use mapping expressions to bind data between steps. The left-hand side is the expected parameter name for the tool or service; the right-hand side is a JSON path expression pointing into the current execution context. For example:

```

input_mapping:
  prompt: $.input.prompt
  duration: $.input.duration
  context: $.steps.retrieve_context.output.context
output_mapping:
  video_title: $.output.title
  video_summary: $.output.summary
  file_id: $.output.file_id

```

Execution Semantics

1. **Sequential execution:** Steps execute in the order they are defined unless transitions specify jumps. Each step starts only after the previous one finishes.
2. **Context accumulation:** The execution context accumulates intermediate outputs. Each step's output is stored under `$.steps.<step_id>.output` and can be referenced by subsequent steps.
3. **Branching:** `local_decision` steps evaluate a boolean and choose the next step. The `on` block for a step can define different targets based on outcome classes (success, failure, partial, requires_confirmation).
4. **Failure propagation:** If a step fails and has no explicit `on.failure` transition, the workflow stops and the execution is marked as `failed`. Partial failures may set status `partial` if some data is recoverable.
5. **Retry logic:** If `retry` is defined, the workflow orchestrator will retry the step upon a classified `tool` or `external_api` error until the `max` count is reached. Validation and permission errors are not retried.
6. **Compensation:** A `saga` step defines compensation actions. If a step with side effects succeeds and a later step fails, the orchestrator will run the compensation steps in reverse order to roll back side effects. Compensation steps themselves may have retries.
7. **Confirmation:** If a tool requires confirmation (`require_confirmation` permission mode), the orchestrator pauses execution and returns a `pending_confirmation` result. Upon confirmation, execution should resume from the specified step.
8. **Timeouts:** Step timeouts are enforced by the tool engine. A timeout is classified as `timeout` error and may be retried if configured.
9. **Completion:** If the last executed step reaches an `end` node with status `success_completed`, the execution result is marked `completed`. If it reaches a `fail_*` end, the result is `failed` or `partial` accordingly.

Error Classes

Workflows classify errors according to the domain error model:

Error Class	Meaning	Retryable
<code>validation</code>	Input or context did not match expected schema.	No

Error Class	Meaning	Retryable
<code>permission</code>	Permission policy did not allow the operation.	Typically no; may allow <code>require_confirmation</code> to continue.
<code>tool</code>	Tool execution failed due to provider error.	Yes, if configured.
<code>workflow</code>	Workflow logic error (e.g. undefined step, bad mapping).	No; fix skill definition.
<code>timeout</code>	Tool did not respond within the allocated time.	Yes, if configured.
<code>external_api</code>	External API returned an unrecoverable error.	Possibly; depends on provider and policy.

Each error should include a `code`, `message`, optional `details`, a `recoverable` flag, and a `suggested_action`. The suggested action could be to retry, adjust input, or update permissions.

Example Workflow

Below is an example of a simplified workflow for the **video director skill**. It follows the manifest example in `skill-manifest-spec.md`.

```
steps:
  - id: validate_prompt
    type: validation
    action: ensurePromptLength
    on:
      success: retrieve_context
      failure: fail_validation

  - id: retrieve_context
    type: tool_call
    tool_id: rag_retrieval_tool
    input_mapping:
      query: $.input.prompt
      topK: 3
    output_mapping:
      context: $.output.items
    on:
      success: generate_video
      failure: normalize_result

  - id: generate_video
    type: tool_call
    tool_id: mock_video_generation_tool
```

```

input_mapping:
  prompt: $.input.prompt
  duration: $.input.duration
  context: $.steps.retrieve_context.output.context
output_mapping:
  title: $.output.title
  summary: $.output.summary
  file_id: $.output.file_id
on:
  success: normalize_result
  failure: normalize_result

- id: normalize_result
  type: service_call
  action: svc.result_normalizer.completed
  on:
    success: success_completed
    failure: fail_workflow

- id: success_completed
  type: end

- id: fail_validation
  type: end

- id: fail_workflow
  type: end

```

This workflow first validates the prompt, retrieves context from RAG, generates a video, then normalizes and returns the result. Errors propagate to appropriate end states.

Notes

- Keep workflows declarative and avoid embedding business logic into step identifiers. Complex logic should be encapsulated in tools or sub-workflows.
- Avoid long chains of retries—prefer to tune individual tool reliability.
- Always consider compensation for side-effecting steps.
- Document each step clearly to aid maintainability and auditing.